Openvswitch without Open vSwitch: The API and its users

Joe Stringer
VMware
USA
joe@ovn.org

Abstract

The `openvswitch` kernel module provides a user-facing API that models a datapath bridge that connects multiple ports and is configured through a single flow table. This flow table allows policies to be written around a wide range of fields and provides actions to manipulate and redirect packets. A primary goal of this design is to allow flexible user-defined policies without complicating the kernel for every use-case; this has been well-aligned with recent industry interest in SDN.

The most well known user of the the `openvswitch` kernel module is ovs-vswitchd, which is a fairly large userspace program that provides an OpenFlow API for programming. However, there are a number of other users and use-cases outside of ovs-vswitchd and SDN for `openvswitch`. This paper presents the `openvswitch` kernel API, shows how it can be directly configured with simpler low-level utilities (eg. ovs-dpctl), and looks at how it compares to similar APIs such as cls_flower. Finally, this paper surveys a variety of open source projects which build upon `openvswitch` without using the Open vSwitch userland utilities.

Keywords

OVS, switch, flow, classifier, action, datapath, API, TC, packet processing, software-defined networking

Introduction

Traditionally, networks have been configured by using distributed protocols where each device in the network builds up its own view of the network and attempts to forward traffic according to that view. While this can often simplify deployment, it can be quite difficult to modify the protocols to improve forwarding efficiency. Software-Defined Networking (SDN) attempts to address this by logically centralizing the network forwarding decisions. For example, B4[12] cited that they were able to improve WAN link utilization by as much as 200-300% by introducing a centralized traffic engineering controller based on SDN. This approach has been shown to be beneficial in several cases, although it does not necessarily replace previous approaches—other efforts have seeked to combine the benefits of traditional and software-defined approaches [11, 2].

The Open vSwitch (OVS) [14] project was born out of this SDN movement, defining a flow-based API for separation of control plane and data plane. Through this API based around packet headers, more granular and therefore efficient network policy could be implemented. Over time, OVS and the OpenFlow [16] specification became increasingly complex to provide powerful control primitives to controllers. Introducing this complexity into the kernel is undesirable; the more complex the protocol, the more potential for bugs; and bugs are more consequential in kernel space. Furthermore, the more that an API is tied to the kernel, the more difficult it is to change it over time.

The level of complexity desired by SDN controllers can be quite high to handle the scale of thousands of virtual switches bridging VMs across thousands of hypervisors[13]. Propagating this complexity from higher layer protocols into the `openvswitch` kernel API (hereafter `openvswitch`) is undesirable from a code complexity and kernel stability perspective. Much of the early work on `openvswitch` was around ensuring compatibility for newer and older userspace daemons and providing generic primitives that could be composed by a userspace daemon to achieve more powerful functionality. This API was eventually accepted into Linux 3.3 [6]. Since the introduction of `openvswitch`, several versions of OpenFlow have been released. Despite the changing protocol definitions above the userspace daemon, no redesign of `openvswitch` has been required to support the new protocols.

This paper seeks to document `openvswitch`, provide enlightenment on the motivating considerations for different aspects of the API design, and describe the projects which use OVS to control packet forwarding behavior in Linux.

Kernel datapath API

Linux provides network policy control to userspace applications through the Netlink protocol [18]. `openvswitch` exposes policy using the Generic Netlink bus by specifying four protocol families: datapath family, virtual port family, flow family, and packet family.

The datapath, virtual port and flow families deal with static configuration of topology and state. Each of these families accepts four operations: NEW, DEL, GET and SET. These commands allow users to create, delete, retrieve and modify state relating to the particular family. `openvswitch` Netlink operations which modify the kernel staterequire `CAP_NET_ADMIN` privileges while GET operations typically may be executed from unprivileged contexts.

The packet family, rather than dealing with static configuration, deals with sending packets to and from userspace applications. The following subsections will go into further detail on each of the openvswitch Netlink families.

All openvswitch Generic Netlink families use a common initial header consisting of an identifying dp_ifindex for the datapath that the operation relates to.

Datapath family

The "ovs_datapath" family allows management of *datapaths* within openvswitch. Each datapath has a name which is unique within the current namespace, and is associated with one or more *virtual ports* and zero or more *flows*. When the datapath is created, a corresponding network device is created with the same name. At datapath creation time, the user must specify a set of one or more Netlink port identifiers (PIDs) to associate with the datapath device. These PIDs are used for upcall handling as described in the next section. Further virtual ports may be attached to the datapath later through the openvswitch virtual port family. Multiple datapaths may coexist at once in a particular namespace, each with different ports attached to them.

A datapath contains a single lookup table (*flow table*) which contains match-action primitives, known as *flows*. At datapath creation time, the flow table is empty. When a packet arrives on a virtual port associated with a datapath, a flow key is extracted from the packet. This flow key is used to look up the flow table for instructions on how to handle the packet–match, and perform a set of actions. Misses in the datapath flow table result in an upcall as described in the next section.

A few statistics are kept per-datapath, including the number of matches within the flow table, the number of missed lookups, the number of flows currently configured, and the number of packets which were lost because they could not be sent to a Netlink PID. Recent implementations also keep track of the number of mask subtables which exist within the flow table and the number of hits on mask subtables. The implementation details relating to these statistics have been previously explained in greater depth here [22].

Virtual port family

The virtual port (*vport*) family allows management of vports within an openvswitch datapath. Each vport has a name and port number that is unique within the given datapath, as well as a port type.

There are several port types, of which the most commonly used is OVS_VPORT_TYPE_NETDEV. This allows a regular Linux network device to be attached to openvswitch in a manner similar to how a device may be attached to a regular Linux bridge. In addition to the netdev vport type, the OVS_VPORT_TYPE_INTERNAL allows configuration of ports that communicate directly with the Linux stack. This port also exists as a netdev, but rather than transmitting via a particular netdev driver it calls up the stack via netif_rx(). These are equivalent to the bridge device that may be created when a user creates a Linux bridge, however in the openvswitch case one or more may be associated with a particular datapath.

For each tunneling protocol that is supported by openvswitch, there is a corresponding port type. At the time of writing, this includes Geneve, VXLAN, GRE in the upstream Linux codebase. Out-of-tree implementations for STT [7] and LISP [8] also exist. Since the advent of Lightweight Tunnels [9], the need for independent vport types for each tunnel protocol is expected to decrease. As of October 2016, the openvswitch maintainers are no longer expanding the vport types to add support for new tunneling protocols [25]. Future implementations should make use of the rtnetlink family to create lightweight tunnel devices for this purpose.

When packets are received and transmitted on a vport, openvswitch keeps track of the number of rx/tx packets and bytes, as well as the number of errors and packets dropped. Errors typically denote unexpected conditions while receiving or transmitting a packet, while drops account for conditions such as transmitting to devices which are currently down or a lack of space available to buffer a packet. Userspace applications may query the statistics of a vport at any time via the OVS_VPORT_CMD_GET operation.

Upcall handling Finally, each vport is configured with one or more Netlink PIDs, just like how the datapath device has one or more Netlink PIDs associated. At port creation time, the userspace application specifies these Netlink PIDs which traffic from that port may be directed to in the case of a miss upcall. The Netlink PID may be 0, meaning that packets should be dropped by default rather than forwarding to the Netlink PID. If the PID is nonzero, packets which do not match a flow in the corresponding datapath's flow table are forwarded to the Netlink socket corresponding to that PID.

More than one Netlink PID may be specified for a particular vport [27]. When a packet misses a lookup in the flow table for a vport with multiple PIDs, the skb->hash is used to select a Netlink PID to send the packet to. This allows the fairness of upcall processing to be handled by preventing a single packet stream from overwhelming a socket and interfering with other upcalls occurring on the same port.

Prior to sending the packet to the Netlink socket for further processing, the skb is normalized, for instance by pushing accelerated VLANs into the packet contents, completing IP checksumming (in the case of CHECKSUM_PARTIAL), and segmenting Generic Segmentation Offload (GSO) frames. Several other pieces of information may also be included in the message along with the packet contents: the kernel-parsed flow key; egress tunnel information; the maximum received unit size if IP defrag was performed; the original packet length if it was truncated; and other user-provided data if the upcall is occurring due to a userspace action.

Flow family

The flow family allows management of *datapath flows* for an openvswitch datapath. Flows are the core packet director structures in openvswitch, which at minimum consist of a set of packet fields and specific values that packets may match, and a set of actions to execute if a packet matches the packet field description.

Flow key This describes each packet header field and metadata field which is relevant to making a match. When packets are forwarded to userspace, the kernel encodes all known attributes about the current packet, including layer 2, 3 and 4 headers, tunnel metadata, and skb metadata such as `skb_mark` or connection tracking state. Each attribute is encoded as a separate TLV when serializing into Netlink format.

This encoding is designed to allow carefully-written userspace applications to support any flow key, regardless of whether the kernel supports more, the same, or fewer flow key fields. A naïve approach to userspace implementation may take the flow key directly from the kernel and use this version of the flow to run through its forwarding logic. However, if the userspace logic requires understanding of a field that is beyond what the kernel can understand, then this may result in incorrect forwarding behavior. Rather, users are suggested to re-parse packets received as upcalls, perform lookup in their own internal structures to determine actions to be executed on this packet, then use the kernel flow key to install flows [10].

Flow mask This consists of the same fields as the flow key, and identifies bits within the fields that are relevant to making the forwarding decision. The flow mask may be omitted from flow creation messages, in which case it acts as an exact match on all bits in each field specified in the flow key.

Flow identifier When a flow is installed into `openvswitch`, the userspace application may use the flow key to identify the field, or it may optionally provide a 128-bit unique flow identifier (UFID). When a flow is created without a UFID, the flow may be subsequently fetched, modified or deleted on the basis of the flow key. The flow key must be specified to apply any operations on that flow. Furthermore, when dumping flows, the identifier of each flow must be encoded to the userspace application. When a flow is created using a UFID, subsequent operations may refer to the flow purely based on the UFID, and omit other flow fields. This allows more efficient encoding and decoding of Netlink messages when other information from the flow is unnecessary for performing the operation.

Actions These specify how the packet should be handled, including packet manipulation; sending the packet to other ports or to userspace; recirculating back to OVS; submitting to other kernel subsystems; cloning; and truncation.

Statistics Each flow contains packet and byte counters which are incremented when a packet matches the flow, as well as the time in milliseconds since this flow was last hit. When a flow is returned to userspace via a GET or DELETE command, the statistics of the flow at the time are sent along with other flow attributes.

Packet family

The packet family describes messages which involve sending a packet between kernel and userspace. The `OVS_PACKET_CMD_EXECUTE` command allows userspace applications to send a packet and a set of actions to the kernel for execution within the datapath context, while the `OVS_PACKET_CMD_MISS` and `OVS_PACKET_CMD_ACTION` commands describe two ways to send packets from the kernel to userspace applications. The miss command is used by default when a packet misses the flow table, and includes the flow key. The action command allows more flexible encoding of information up to userspace, and is triggered by the `OVS_ACTION_ATTR_USERSPACE` action. This action may include a range of other information, including fields such as the egress tunnel key or the actions list currently being processed.

Notable openvswitch improvements

Over time to improve performance and make `openvswitch` more useful for advanced use cases, several notable improvements to `openvswitch` have been introduced. This section briefly describes some of these features.

Megaflows

Prior to the introduction of masked lookup, the `openvswitch` flow table was prone to poor worst-case performance when many different connections were handled by `openvswitch`, as each new connection would trigger a separate upcall. Linux 3.12 introduced support for masked flow lookups in `openvswitch`, or *megaflows*. This design modifies the lookup to iterate through a list of masks, then for each mask, apply the mask to the key to assemble a masked key, which is used for lookup in the flow table. No ordering is defined between the masks, so it is up to the userspace implementation to prevent overlapping flows. Note that the external kernel API still presents a single logical flow table. These masks allow `openvswitch` users to specify the granularity of the flow key that should be used for each flow match. In some cases, this feature improved connection setup performance by as much as 300% while reducing CPU and memory usage in the same case from 90% to 20% [22].

Recirculation

While the userspace representations of the desired forwarding state have always been complex, the early versions were able to compile down multiple lookups of a flow into a single lookup in `openvswitch`. Newer use cases have made it more difficult to perform a single packet classification in the kernel to determine forwarding behavior. For example, if an MPLS packet arrives to `openvswitch` and the packet is extracted, the packet contents beyond the MPLS header is not known. By design, this state is only available to the MPLS control plane. Thus, to match on MPLS header in the datapath, pop the header off to reveal another protocol, then perform actions based on the popped protocol, a second packet classification is required. The `recirculate` action provides a method for one flow in `openvswitch` to send a packet back to the beginning of datapath processing to re-extract the flow key to allow further processing. Several features now make use of recirculation, such as bonding, MPLS support and connection tracking.

Lightweight Tunnels

Nicira NVP was designed to scale to thousands of hypervisors, connecting each pair together via a tunnel [13]. This represents some challenges when dealing with the net devices in Linux, as these were not designed with such scale in mind. Configuring so many devices was slow, and for tunnel devices much of the structure was unnecessary. In practice this increased the CPU and memory footprint of tunnels in a way that was not scalable. To allow the configuration of thousands of tunnels without imposing performance degradation from managing thousands of net devices, `openvswitch` introduced the notion of a flow-based tunnel device. More recently, this work was expanded to be used from other subsystems: Lightweight Tunneling [9].

NetFilter integration

Traditionally, `openvswitch` focused on stateless matching on packets. To implement functionality such as a firewall, several approaches were explored including matching on TCP flags and reactive learning of new flows to allow traffic [20]. However, such approaches had limitations in terms of how comprehensively they could detect and block traffic. In Linux 4.3, `openvswitch` introduced integration with the NetFilter subsystem to allow stateful operations to be executed from OVS. This has allowed `openvswitch` to be used for implementing stateful firewalls, stateful NAT and load balancers. Several cloud management softwares including OpenStack, Kubernetes and Mesosphere now make use of `openvswitch` stateful functionality through different projects[5, 30]. This work allowed the performance of the OpenStack security group use case to be significantly improved [1].

Users of openvswitch

There are several well-known open source users of `openvswitch`, providing higher level abstractions for network forwarding behavior. The users are implemented in a spread of languages, including C, Java and Go. In some cases, the projects provide a small shim utility that operates to translate commandline arguments directly into the equivalent kernel API calls.

Broadly, the datapath and vport configuration tends to be handled by userspace daemons when topology changes occur. When virtual switches are configured or ports are added or deleted, daemons will typically push this configuration state down to `openvswitch` as soon as they are aware of new topology.

Most users of `openvswitch` follow a common model regarding flow installation. Although it is possible to preconfigure flows directly from userspace and disable the upcall mechanism, users tyically perform a form of Just-In-Time (JIT) assembly of kernel flows as traffic arrives. The daemons hold an abstracted representation of the desired forwarding state in the userspace and subscribe to packet notifications from kernel. When new traffic is received in the kernel, the packet is forwarded to the userspace component which simulates the full desired packet forwarding behavior and generates a flow to handle similar traffic in future. Previous investigations have asserted that for complicated forwarding pipelines, this reactive model often out-performs the equivalent fixed-function implementation [22].

In the worst case, this reactive model may degrade, resulting in overwhelming the userspace daemon with more packets than it is able to handle. A similar concern is that a malicious user could send a variety of traffic on a particular port and prevent other users from connecting through the port. These concerns have been largely mitigated by a combination of the megaflows feature and support for multiple netlink PIDs per port. Future work also seeks to ratelimit upcall channels to allow userspace daemons to further protect themselves in these cases.

Open vSwitch daemon

The Open vSwitch daemon (Open vSwitch) is the most well-known user of `openvswitch`, and is frequently used in a variety of use cases such as for the virtual switch on hypervisors and the forwarding agent on enterprise switches and Customer-Premises Equipment (CPE). It is used to implement virtual networks for several commercial SDN solutions across multiple platforms including Linux and Windows Hyper-V.

Open vSwitch provides an abstraction above `openvswitch` to allow controllers to define packet forwarding behavior through the OpenFlow and OVSDB [21] protocols. These protocols allow ephemeral state and persistent configuration to be applied to the forwarding logic.

There are several derivatives of the official Open vSwitch which are tailored to particular use cases or vendor integration [15, 19]. Beyond Open vSwitch and its derivatives, there are several other users of `openvswitch`. The following subsections describe briefly some of the ways that `openvswitch` is used, and the goals of those projects.

CLI tools

Open vSwitch [14] and go-odp [28] provide commandline utilities for reading and manipulating `openvswitch` configuration. They provide commandline access to the datapath, vport and flow families. The most common use case for these utilities is for online debugging when `openvswitch` is used in a more complex environment.

Commandline utilities can often be useful for development and debugging in a simplified environment. Userspace managers of `openvswitch` will often include intricate code to abstract the kernel API to a more generic network view, which can obscure the kernel API behavior. Developing and testing directly against the kernel API using tools such as `ovs-dpctl` can reduce the effort required to prototype new functionality.

Listing 1 contains an example of configuring an `openvswitch` datapath with two ports and creating flows to allow ICMP communication between the ports. The presentation for this paper will provide more detailed examples on `ovs-dpctl` usage for common debugging and development use cases. If errors occur running `ovs-dpctl`, then `openvswitch` will log messages to `dmesg` for further investigation.

Listing 1: Example ovs-dpctl usage

```
ovs-dpctl add-dp myDP
ovs-dpctl add-if myDP eth0
ovs-dpctl add-if myDP eth1
ovs-dpctl show
ovs-dpctl add-flow "in_port(1),eth(),
    eth_type(0x806),arp()" 2
ovs-dpctl add-flow "in_port(2),eth(),
    eth_type(0x806),arp()" 1
ovs-dpctl add-flow "in_port(1),eth(),
    eth_type(0x800),ipv4(proto=1)" 2
ovs-dpctl add-flow "in_port(2),eth(),
    eth_type(0x800),ipv4(proto=1)" 1
ovs-dpctl dump-flows
```

MidoNet

MidoNet [17] aims to provide virtual network abstractions above `openvswitch` which are useful for implementing Infrastructure-as-a-Service (IaaS) clouds. Users such as OpenStack seek to configure distributed L2 and L3 forwarding, load balancing and security groups to apply to hypervisors within the cloud. To achieve this, MidoNet configures a mesh of VXLAN tunnels across the configured hypervisors and programs `openvswitch` flows to implement the above networking features.

MidoNet is built in a fully distributed manner which involves an agent running on each node that manages the network configuration. It highlights the ability to perform distributed load balancing and floating IP address support. MidoNet also makes use of other subsystems in conjunction with OVS, such as Hierarchical Token Buckets (HTB) to guarantee network service levels for multi-tenant environments.

Weave Net

Weaveworks maintains a Go language binding for `openvswitch` called go-odp [28], and uses this implementation for its container network management software, Weave Net [30]. Weave aims to make it easy to set up a virtual network across several hosts and connect containers on each of the hosts together.

Weave Net creates a L2 overlay network for containers by establishing VXLAN tunnels among the hosts and installing `openvswitch` routing flows. From an end-user perspective, Weave Net does not require any configuration besides a subset of hosts also running Weave Net. Through a distributed gossip protocols, it can transitively discover other hosts participating in the cluster via Weave Mesh [29]. Weave Net also integrates OVS with other in-kernel constructs such as XFRM for encryption.

Indigo

Project Floodlight maintains Indigo Virtual Switch (IVS) [3], another virtual switch targeted to datacenter use cases. Building upon the Indigo OpenFlow agent framework, IVS aims to provide a portable, extensible virtual switch that may be configured via OpenFlow. IVS is used to implement underlay connectivity and monitoring for hypervisors [4].

Related Work

The design and implementation of Open vSwitch [22] describes in detail the considerations that have influenced the architecture of the `openvswitch` forwarding model. The basis for the model starts with flow-based traffic classification using tuple space search [26] which has been optimized for lookup performance. The paper describes in depth how complex flows with overlapping masks and multiple priorities across multiple tables may be mapped down into a single table of non-overlapping flows in kernel space. This work has influenced the OVS kernel API in a few ways, most notably in terms of the "megaflow" feature.

Previous work has explored whether the `openvswitch` forwarding model could be integrated into the Linux Traffic Control (TC) subsystem [24]. Since that work, cls_flower was merged into Linux [23] and the community has actively attempted to expand its functionality to reach parity with `openvswitch`. This work specifically aims to implement the flow key functionality from the `openvswitch` flow family. However, it does not address the remaining aspects of the `openvswitch` API.

There remains several missing features to the cls_flower implementation. Some features are simpler than others—for instance, the lack of matching on TLVs or flags within tunnel metadata, MPLS, TCP flags, and neighbour discovery message types. Beyond the flow key, there is no integration with stateful services via the netfilter subsystem, which is important for several `openvswitch` users. Furthermore, there is currently no equivalent to the `openvswitch` packet family which allows the exchange of combined packet and datapath state with a userspace process. While the aforementioned work has the laudable goal of reducing code duplication and consolidating kernel APIs, it remains to be seen whether this can be achieved without complicating the clearly defined `openvswitch` forwarding model.

Conclusion

This paper described the origins and motivation for `openvswitch` as a way to provide flexibility to network operators to guide packet forwarding behavior based on logically centralized flow configuration. This approach is used widely by a variety of different software projects which seek to define Linux forwarding policy in terms of flows. Over time, this has increased the usefulness of the Linux kernel while minimizing the kernel code complexity required to implement elaborate network forwarding policies.

Acknowledgments

The authors would like to acknowledge Justin Pettit for his guidance on writing this paper, Martynas Pumputis for providing context around WeaveNet, and Ben Pfaff, Hannes Sowa and Yingying Ren for providing review feedback.

Author Biography

Joe Stringer is a software engineer interested in networks and related fields, in particular SDN. He is actively engaged in the open-source community on network software projects including Open vSwitch and the Linux networking stack.

References

[1] Ajo, M. A.; Graf, T.; Lazzaro, I.; and Pettit, J. Taking Security Groups to Ludicrous Speed with Open vSwitch. https://www.openstack.org/videos/vancouver-2015/taking-security-groups-to-ludicrous-speed-with-open-vswitch.

[2] Bailey, J., and Stuart, S. 2016. Faucet: Deploying sdn in the enterprise. *ACM Queue* 14 Issue 5:54–68.

[3] Big Switch Networks. Indigo Virtual Switch. http://www.projectfloodlight.org/indigo-virtual-switch/.

[4] Big Switch Networks. Switch Light. http://www.bigswitch.com/products/switch-light.

[5] Bryant, R.; Mestery, K.; and Pettit, J. OVN: Open Virtual Network for Open vSwitch. http://openvswitch.org/support/slides/OVN-Vancouver.pdf.

[6] Corbet, J. Routing Open vSwitch into the mainline. https://lwn.net/Articles/469775/.

[7] Davie, B., and Gross, J. 2016. A Stateless Transport Tunneling Protocol for Network Virtualization (STT). Internet-Draft draft-davie-stt-08, Internet Engineering Task Force. Work in Progress.

[8] Farinacci, D.; Fuller, V.; Meyer, D.; and Lewis, D. 2013. The Locator/ID Separation Protocol (LISP). RFC 6830.

[9] Graf, T. Lightweight & flow based tunneling. https://lwn.net/Articles/650778/.

[10] Gross, J.; Zhou, A.; and Stringer, J. Open vSwitch Datapath Development Guide. http://docs.openvswitch.org/en/latest/topics/datapath/.

[11] Gupta, A.; Vanbever, L.; Shahbaz, M.; Donovan, S. P.; Schlinker, B.; Feamster, N.; Rexford, J.; Shenker, S.; Clark, R.; and Katz-Bassett, E. 2014. Sdx: A software defined internet exchange. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, SIGCOMM '14, 551–562. New York, NY, USA: ACM.

[12] Jain, S.; Kumar, A.; Mandal, S.; Ong, J.; Poutievski, L.; Singh, A.; Venkata, S.; Wanderer, J.; Zhou, J.; Zhu, M.; Zolla, J.; Hölzle, U.; Stuart, S.; and Vahdat, A. 2013. B4: Experience with a globally-deployed software defined wan. *SIGCOMM Comput. Commun. Rev.* 43(4):3–14.

[13] Koponen, T.; Amidon, K.; Balland, P.; Casado, M.; Chanda, A.; Fulton, B.; Ganichev, I.; Gross, J.; Ingram, P.; Jackson, E.; Lambeth, A.; Lenglet, R.; Li, S.-H.; Padmanabhan, A.; Pettit, J.; Pfaff, B.; Ramanathan, R.; Shenker, S.; Shieh, A.; Stribling, J.; Thakkar, P.; Wendlandt, D.; Yip, A.; and Zhang, R. 2014. Network virtualization in multi-tenant datacenters. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*, 203–216. Seattle, WA: USENIX Association.

[14] Linux Foundation. Open vSwitch. http://openvswitch.org/.

[15] Linux Foundation. OpenSwitch. http://www.openswitch.net/.

[16] McKeown, N.; Anderson, T.; Balakrishnan, H.; Parulkar, G.; Peterson, L.; Rexford, J.; Shenker, S.; and Turner, J. 2008. Openflow: Enabling innovation in campus networks. *SIGCOMM Comput. Commun. Rev.* 38(2):69–74.

[17] Midokura. MidoNet. http://midonet.org/.

[18] Neira-Ayuso, P.; Gasca, R. M.; and Lefevre, L. 2010. Communicating between the kernel and user-space in linux using netlink sockets. *Softw. Pract. Exper.* 40(9):797–810.

[19] NoviFlow. vSwitch Offload. http://noviflow.com/solutions/vso/.

[20] Pettit, J., and Graf, T. Stateful connection tracking and stateful NAT. http://openvswitch.org/support/ovscon2014/17/1030-conntrack_nat.pdf.

[21] Pfaff, B., and Davie, B. 2013. The Open vSwitch Database Management Protocol. RFC 7047.

[22] Pfaff, B.; Pettit, J.; Koponen, T.; Jackson, E.; Zhou, A.; Rajahalme, J.; Gross, J.; Wang, A.; Stringer, J.; Shelar, P.; et al. 2015. The design and implementation of open vswitch. In *12th USENIX symposium on networked systems design and implementation (NSDI 15)*, 117–130.

[23] Pírko, J. introduce programable flow dissector and cls_flower. https://lwn.net/Articles/642028/.

[24] Pírko, J. 2015. Implementing open vswitch datapath using tc. In *netdev 0.1*.

[25] Shelar, P. openvswitch: use ip-gre tunnel rather than gretap tunnel. https://www.spinics.net/lists/netdev/msg400135.html.

[26] Srinivasan, V.; Suri, S.; and Varghese, G. 1999. Packet classification using tuple space search. *SIGCOMM Comput. Commun. Rev.* 29(4):135–146.

[27] Wang, A. datapath: Allow each vport to have an array of 'port_id's. https://mail.openvswitch.org/pipermail/ovs-dev/2014-March/281192.html.

[28] Weaveworks. go-odp. https://github.com/weaveworks/go-odp.

[29] Weaveworks. Weave Mesh. https://github.com/weaveworks/mesh.

[30] Weaveworks. Weave Net. https://www.weave.works/products/weave-net/.